Scala: a Cross-Platform Language

How to build applications that span multiple platforms

Gianluca Aguzzi gianluca.aguzzi@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Alma Mater Studiorum – Università di Bologna

Talk @ Paradigmi di Progettazione e Sviluppo

25/05/2026

Cross-Platform Development: Background

Definition

Cross-platform software refers to applications designed to run seamlessly across *multiple target computing platforms* (different operating systems, architectures, or runtimes) using a unified codebase

Motivations & The Classical Paradigm

- **Terminology**: Also referred to as *platform-agnostic* or *multi-platform*
- **The Core Motivations**: Drastically reduce development costs, eliminate code duplication, and ensure business logic remains synchronized across platforms
- **The Classical Paradigm**: The Java Virtual Machine (JVM) pioneered this with the famous model: *Write once, run everywhere* (using portable bytecode)

Modern Cross-Platform: From VMs to Transpilation

The Modern Shift: Target Compilation

Instead of running a heavy virtual machine everywhere, modern languages compile directly to target environments:

- **Transpilation**: Compiling a high-level language directly into another high-level language (e.g., Scala/Kotlin/Dart $\rightarrow$ JavaScript)
- **Ahead-of-Time (AOT) Compilation**: Compiling directly to native machine code via toolchains like LLVM (e.g., Scala Native, Rust, Go)

Universal Runtimes & Polyglot Coexistence

- **New universal runtimes**: High-performance browser JIT engines (JavaScript) and WebAssembly (Wasm) runtimes are the new portable execution environments
- **Polyglot UI SDKs**: Shared core business logic in a single language, but interfacing with highly platform-specific SDKs for native UI rendering

Frameworks & Languages for Cross-Platform Development

Frameworks (UI oriented)

- Flutter 🔗: *an open-source framework by Google for multi-platform native apps (originally for Android, iOS, and Windows, now supporting Linux and macOS as well)*
  - Motto: *Build apps for any screen*
  - Pretty recent (2017)
- Jetpack Compose Multiplatform 🔗: *Kotlin's declarative UI framework for sharing UIs across multiple platforms*
  - Supports Android, iOS, Desktop, and Web
  - Based on Android's Jetpack Compose
- React Native 🔗: *React Native brings React's declarative UI framework to iOS and Android*
  - Motto: *Learn once, write anywhere*

Languages (Multi-platform)

- Kotlin: supports several target runtimes thanks to **Kotlin Multiplatform** projects 🔗
  - Introduced with Kotlin 1.2 (alpha), became experimental in 1.4, and is now stable (as of 2023)
  - Supports JS, JVM, & Native platforms (iOS, LLVM, Android, ...)
- Dart: Google's programming language that compiles to multiple platforms 🔗
  - Compiles to native ARM & x64 machine code for mobile, desktop, and server
  - Compiles to JavaScript for web applications
  - Primary language for Flutter framework
- **Scala** (focus of today): targets different platforms using *compiler & SBT plugins* 🔗:
  - Scala.js 🔗: stable version to compile Scala to JS (released in 2014!!)
  - Scala Native 🔗: alpha version supporting native applications

Scala Cross-Platform

Benefits

- **Shared codebase**: application logic is *shared* among several targets, avoiding *code duplication* & *error propagation*
- **Full-stack orientation**: using a programming language that targets multiple platforms enables writing both backend and client with the same language
- **Access to libraries & SDKs**: a multi-platform language can leverage platform-specific libraries that were not originally designed for it (e.g., TensorFlow from Scala!!)

Use cases

- Cross-platform library (e.g., Cats 🔗)
- Web application development (Scala.js)
- Cloud lambda functions/container (native)
- Shared code for Android & iOS apps
- **N.B.** the "target code to Scala" use case is typically not considered

Scala.js & Scala Native Configuration: SBT is your friend!

- The Scala compiler (scalac) does not natively output JavaScript or native executables
- You have to enable *plugins* via SBT configuration

project/plugins.sbt

```
addSbtPlugin(
 "org.scala-js" % "sbt-scalajs" % "1.21.0"
)
// for native
addSbtPlugin(
 "org.scala-native" % "sbt-scala-native" % "0.5.10"
)
```

build.sbt

```
import sbt.Keys.scalaVersion
ThisBuild / scalaVersion := "3.3.5"
name := "scala-js-native"

// Multi project configuration with SBT
// Each project call configuration a sub
// project with the name given in the val
val js = project.settings(
  scalaJSUseMainModuleInitializer := true
).enablePlugins(ScalaJSPlugin)

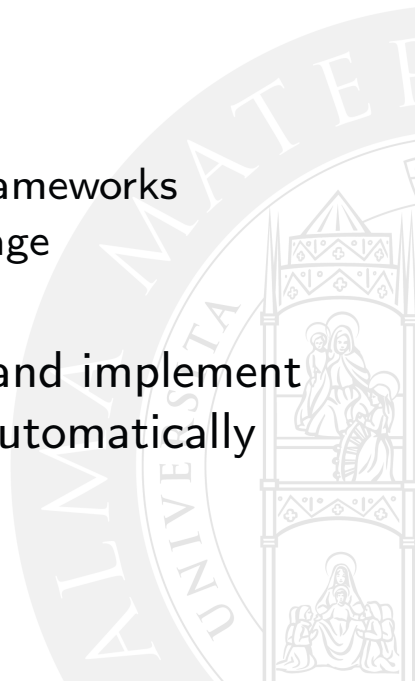
val native = project
  .settings(
    nativeConfig ~= { _.withBaseName("native-out") }
  )
  .enablePlugins(ScalaNativePlugin)
```

Architectural Patterns for Multi-Platform: Clean Architecture

The Core Principle: Ports & Adapters

Decouple core business logic from platform-specific APIs and target runtimes using **Clean / Hexagonal Architecture**

- **The Pure Core (Platform-Agnostic)**:
  - Contains domain models, business logic, validation rules, and pure functions
  - Written in standard Scala, targeting the *Shared* project
  - Uses only cross-platform libraries (e.g., Cats, Circe)
- **Platform Adapters (Target-Specific)**:
  - JVM target: Database engines (JDBC, Doobie), enterprise server frameworks
  - JS target: DOM management (Laminar, Tyrian), local browser storage
  - Native target: Direct memory, POSIX file APIs, hardware C-interop
- **Interface Segregation**: Define generic traits in the shared module, and implement them on each platform. SBT handles cross-linking and packaging automatically



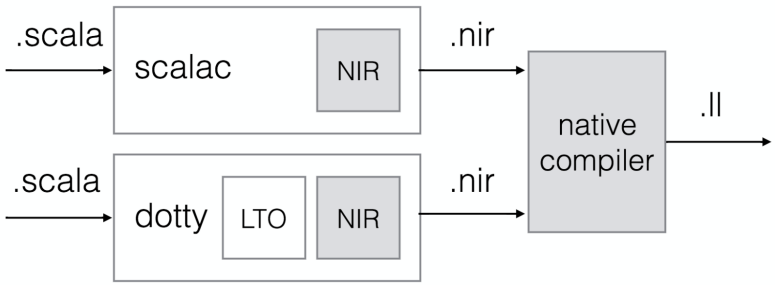
The Scala 3 Multi-Platform Era

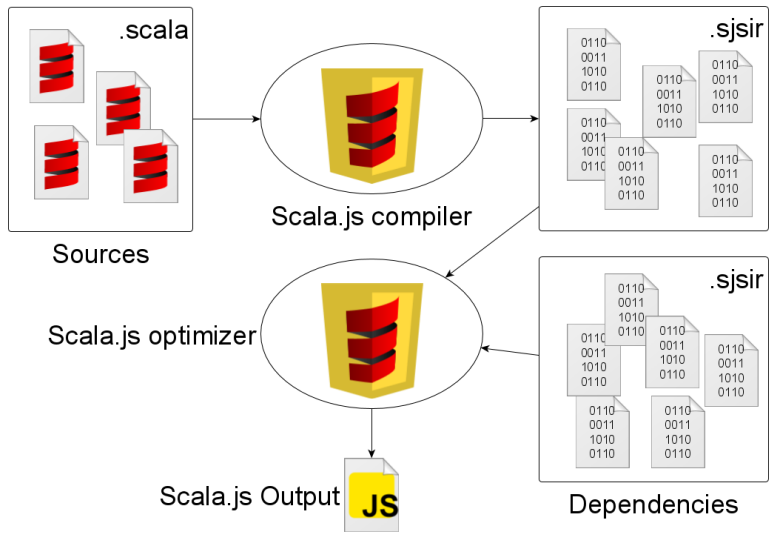
Scala 3: A Unified Compiler Experience

- **First-class target support**: Scala 3 has native, out-of-the-box support for both Scala.js (and Scala Native)
- **Language unification**: Advanced features like *givens, extension methods, union types, and enums* compile seamlessly to JS and LLVM Native without performance penalties
- **Improved Developer Experience**:
  - **Metals (IDE)** 🔗: Full support for cross-platform projects with instant context-switching between JVM, JS, and Native target environments
  - **Tasty (Typed Abstract Syntax Trees)**: Library binary compatibility is significantly enhanced across targets

How Scala Native & Scala.js Work [1]

- The compiler takes plain Scala code
- Using a compiler plugin, it produces an intermediate representation (IR) that contains platform-dependent aspects
- Using the IR, the compiler performs optimization, linking, and dependency management





[1] Sébastien Jean R Doeraene. "Cross-Platform Language Design". en. In: (2018). DOI: 10.5075/EPFL-THESIS-8733. URL: http://infoscience.epfl.ch/record/256862

Scala.js compiles to WebAssembly (WasmGC)

The WebAssembly Frontier

- **Experimental Wasm target** (introduced in Scala.js 1.17+ / 1.21.0): Compiles Scala directly to standard WebAssembly!
- **Technology**: Leverages **WasmGC** (WebAssembly Garbage Collection) and **Exception Handling** proposals to achieve near-native performance inside modern browsers and Node.js
- **Benefits**: Substantially faster execution times for compute-heavy workloads, and reduced JIT warm-up times

Enabling Wasm in build.sbt

```
scalaJSLinkerConfig := {
  scalaJSLinkerConfig.value
    .withExperimentalUseWebAssembly(true)
    .withModuleKind(ModuleKind.ESModule)
}
```

Caveats: Challenges & Limitations

The Trade-Offs of Cross-Compilation

- **Limited JVM ecosystem access**: In pure cross-platform projects, you **cannot** use the general JVM ecosystem (e.g., threads, file systems) unless a cross-platform facade or reimplemented library exists
- **Scala Native is still maturing**: Unlike Scala.js, the Scala Native toolchain and ecosystem are still relatively limited:
  - Smaller library support, longer compile times (LLVM linking is heavy)
  - Standard library coverage is incomplete, and garbage collection tuning is complex
- **Large bundle sizes**: Compilation output must include parts of the Scala runtime and standard library (e.g., Scala.js bundles easily reach $\sim$1 MB before tree-shaking / minification)

Bridging the Ecosystems: The Need for Facades

The Interoperability Challenge

How does Scala code talk to native runtime APIs (JS in browsers, C in OS)? We need type declarations that bridge Scala types to JS globals or C headers

- **Scala.js Facades**: Written as standard Scala objects/classes annotated to tell the compiler they represent JavaScript APIs
- **Scala Native Bindings**: Require linking to C libraries via `@extern` and low-level pointers (`Ptr[Byte]`)

Scala.js Facade (Browser Alert)

```
@js.native
@JSGlobal
object window extends js.Object {
  def alert(msg: String): Unit = js.native
}
```

Scala Native C-Binding (Math)

```
@link("m")
@extern
object libm {
  def sqrt(d: Double): Double = extern
}
```

Scala.js Interoperability: Core Concepts

How Scala Meets JavaScript

Scala.js compiles standard Scala code directly to JavaScript. To interact with JavaScript ecosystems (like NPM modules or DOM APIs), it provides an expressive type system to model JavaScript's dynamic nature with strong compile-time guarantees

The Three Pillars of Scala.js Interop

- **Facades (`@js.native`)**: Pure type definitions for existing JS APIs. They do not generate JS code, but instruct the compiler how to emit correct JS calls
- **JS Conversions (`JSConverters`)**: Implicit and explicit conversions to translate Scala collections (`Seq`, `Map`, `Option`) to JS collections (`js.Array`, `js.Dictionary`, `js.UndefOr`)
- **Exports (`@JSExportTopLevel`)**: Annotating Scala classes/objects to expose them to external JavaScript code, enabling bi-directional invocation

ScalablyTyped: Automated Facade Generation

The Typing Goldmine: TypeScript Definitions

Writing facades manually for large libraries like React, Three.js, or Lodash is practically impossible. But TypeScript has **DefinitelyTyped** (`.d.ts`), which contains type definitions for almost every NPM package

How ScalablyTyped Works

A compiler and SBT plugin that parses these TypeScript definitions and:

- Automatically generates rich, type-safe, idiomatic Scala.js facades
- Translates TS union types to Scala 3 union types, generics, and functions
- Unlocks 100% type safety and IDE autocomplete for the entire NPM ecosystem!

Scala.js vs. Scala Native: Ecosystem Maturity

Scala.js: Production-Ready & Highly Mature

- Stable since 2014; near-perfect compatibility with standard Scala features
- Garbage collection is delegated to the host browser (extremely fast, advanced V8/SpiderMonkey GCs)
- Rich ecosystem of pure Scala reactive frameworks (Laminar, Tyrian)

Scala Native: High-Potential but Still Maturing

- Excellent for standalone executables, instant startup, and tiny memory footprint
- But: LLVM compilation is very slow, standard library coverage has gaps, and GC (Boehm/Immix) is still less mature than JVM or browser GCs
- Binding complexity: No automated tool like ScalablyTyped exists for C/C++ header files, requiring verbose manual glue code or custom generator scripts

Say Goodbye to Verbose Facades & JavaScript Boilerplate

We no longer need to write raw facades for React or Vue. The Scala 3 community has created modern, reactive, type-safe web frameworks:

- **Laminar** 🔗:
  - A pure Scala, highly-optimized reactive UI library using Observables
  - No external JS library dependencies (it targets the DOM directly)
  - Extremely popular for modern, highly responsive Scala.js web applications
- **Tyrian** 🔗:
  - Developed by the Typelevel ecosystem for Scala 3
  - Follows the elegant Elm Architecture (Model-View-Update)
  - Integrates natively with **Cats Effect** for asynchronous and effectful operations



Multi-Platform Setup

- Another SBT plugin to configure multiple platform projects: **sbt-crossproject**
- Enforces a standard project structure (configurable via different *flavors*)
  - `CrossType.Pure`: pure cross-platform projects (e.g., libraries), where all code is placed under `src/`
  - `CrossType.Full`: projects with platform-specific code (e.g., UI)
    - `shared`: pure multi-platform code (e.g., data structures, interfaces, ...)
    - `js`, `jvm`, and `native`: contain platform-specific application code (e.g., library usage, GUIs, ...)

project/plugins.sbt

```
val crossOrg = "org.portable-scala"
val crossNative = "sbt-scala-native-crossproject"
val crossJs = "sbt-scalajs-crossproject"
addSbtPlugin(crossOrg % crossJs % "1.3.2")
addSbtPlugin(crossOrg % crossNative % "1.3.2")
addSbtPlugin(
  "org.scala-js" % "sbt-scalajs" % "1.21.0"
)
addSbtPlugin(
  "org.scala-native" % "sbt-scala-native" % "0.5.10"
)
```

build.sbt

```
crossProject(JSPlatform, NativePlatform, JVMPlatform)
  .crossType(CrossType.Full) // or CrossType.Pure
// common settings, use %%% for cross library
  .settings(
    libraryDependencies ++= Seq(
      "org.scalatest" %%% "scalatest" % "3.2.19"
    )
  )
  .jsSettings()
  .nativeSettings()
  .jvmSettings() // standard "scala" world
```

The Triple-Target Ecosystem

Write Once, Publish Triply

Most major libraries in the Scala ecosystem are published simultaneously for the JVM, JavaScript, and Native targets. You can share your business logic, data validation, and test suites across all of them!

- **Typelevel Core**: *Cats* (functional programming), *Cats Effect* (async runtime), and *FS2* (streaming) have first-class support on JVM, JS, and Native
- **Communication & APIs**: *Http4s* (HTTP client/server) and *Tapir / Smithy4s* allow sharing API definitions and serializing types across targets
- **JSON / Serialization**: *Circe*, *uPickle* support pure cross-compilation
- **Testing**: *MUnit* and *Weaver* provide identical testing frameworks on all targets

Examples

Library Facade: TensorFlow.js

- Wrap Google's high-performance **TensorFlow.js** library in a clean, memory-safe, pure-Scala API

Cross-target Application: Winning Four

- Share core business logic in a pure-Scala shared project while writing platform-specific GUIs (Web, Android)

Full-Stack Application: Todo Service

- Share data validation and schemas seamlessly between a JVM backend and a Scala.js frontend

Scala Boids

- Simulating flocking behavior with a shared codebase across multiple platform targets

Library Facade: TensorFlow.js (The Philosophy)

The Goal: A Pure Scala API

Writing facades allows us to consume rich JavaScript libraries while exposing a clean, idiomatically typed API to our clients

Hiding the Interoperability Boilerplate

In a clean architectural style, our public interface should hide:

- All Scala.js interop annotations and custom JavaScript types
- Implicit conversions between Scala standard collections and JavaScript arrays
- Possibly, low level details like memory management (e.g., `dispose()` calls for tensors)

Writing a Facade: The Interop Keywords

Main Scala.js Interop Concepts

To declare JavaScript APIs in Scala, we use a set of specialized keywords, types, and annotations:

- `@JSImport`: Instructs the Scala.js linker to import a packaged NPM dependency or ES module
  - *Example:* `@JSImport("@tensorflow/tfjs", JSImport.Namespace)` imports the global `tf` namespace from the `@tensorflow/tfjs` package
- `@js.native`: Tells the compiler that the class, object, or trait has a native JavaScript implementation. No Scala method body bytecode is emitted
- `js.Object`: The base trait for any class/object representing a native JavaScript object
- `js.UndefOr[A]`: Represents a value that may be a type `A` or `undefined` in JavaScript (similar to Scala's `Option`)

Bridging Standard Collections with `JSConverters`

To exchange data between Scala and JS seamlessly, we use `scala.scalajs.JSConverters.*`:

- **Implicit Conversions**: Converting Scala sequences/maps to JavaScript arrays and dictionaries via extension methods (e.g., `seq.toJSArray`)
- **Asynchronous Futures**: Mapping JavaScript native promises to Scala standard Futures (via the `.toFuture` extension method on `js.Promise[A]`)

ScalablyTyped Example: SBT Configuration

Automated Facade Generation with ScalablyTyped

Instead of manually writing facades, ScalablyTyped automatically converts TypeScript definition files (`.d.ts`) into rich, type-safe Scala.js interfaces

Enabling ScalablyTyped in build.sbt

```
lazy val scalablytyped = (project in file("scalablytyped"))
  .enablePlugins(ScalaJSBundlerPlugin, ScalablyTypedConverterGenSourcePlugin)
  .settings(name := "scala-js-facade-scalablytyped",
    scalaJSUseMainModuleInitializer := true,
    Compile / npmDependencies += "@tensorflow/tfjs" -> "4.22.0",
    stOutputPackage := "customtypings")
```

Idiomatic JavaScript in Pure Scala

Instead of manually writing facades, ScalablyTyped automatically converts type definitions, making NPM packages feel like native Scala libraries

Safe Usage of the Generated Facade

```
import scala.scalajs.js
import customtypings.tensorflowTfjs.mod as tf
val data = js.Array(1.0, 2.0, 3.0, 4.0)
val tensor = tf.tensor1d(data)
val squared = tensor.square()
tensor.dispose()
```

What's happening under the hood? ScalablyTyped parses the TypeScript definitions and generates Scala facades that:

- **Package-Mapped Namespaces**: TypeScript modules and ES imports are automatically bound to standard Scala packages (e.g., `customtypings.tensorflowTfjs`)
- **Type-Safe Invocations**: TypeScript's structural typing, union types, and callbacks are compiled into Scala 3 union types and functions, verified at compile-time

ScalablyTyped: Build Styles & Code Flavors

1. Build & Dependency Integration Styles

How ScalablyTyped integrates into your project build pipeline:

- **Managed Bundler** (`ScalablyTypedConverterPlugin`): Centralized in SBT; reads `npmDependencies` and compiles facades on-the-fly
- **External NPM** (`ScalablyTypedConverterExternalNpmPlugin`): Perfect for modern tools like **Vite**; uses your local `package.json` and `node_modules` for a faster build loop

2. Code Output Flavors (`stFlavour`)

Tells the generator how to adapt TypeScript components for specific Scala UI wrappers:

- `Flavour.Normal` (Default): Standard Scala.js facades representing raw JS objects (e.g., Three.js, TensorFlow.js)
- `Flavour.Slinky`: Emits facades fully integrated with Slinky (ergonomic React wrapper)
- `Flavour.ScalajsReact`: Emits facades matching Japgolly's scalajs-react (highly functional/Cats)

Scala: a Cross-Platform Language

How to build applications that span multiple platforms

Gianluca Aguzzi gianluca.aguzzi@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Alma Mater Studiorum – Università di Bologna

Talk @ Paradigmi di Progettazione e Sviluppo

25/05/2026

References I

[1] Sébastien Jean R Doeraene. "Cross-Platform Language Design". en. In: (2018). DOI: 10.5075/EPFL-THESIS-8733. URL: http://infoscience.epfl.ch/record/256862.

[2] *Kotlin Multiplatform | Kotlin*. https://kotlinlang.org/docs/multiplatform.html. (Accessed on 05/22/2022).

[3] *React Native · Learn once, write anywhere*. https://reactnative.dev/. (Accessed on 05/22/2022).

[4] *Scala Native — Scala Native 0.4.5-SNAPSHOT documentation*. https://scala-native.readthedocs.io/en/latest/. (Accessed on 05/22/2022).

[5] *Scala.js*. https://www.scala-js.org/. (Accessed on 05/22/2022).

[6] *Showcase - Flutter apps in production*. https://flutter.dev/showcase. (Accessed on 05/22/2022).